

LAPORAN PRATIKUM
ALGORITMA DAN PEMROGRAMAN

SORTING 1

Disusun Oleh:

Khairun Nisa

2511532015

Dosen Pengampu:

DR. WAHYUDI, S.T, M.T

Asisten Pratikum:

Sherly Sukmadira



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas terselesaikannya laporan praktikum Algoritma dan Pemrograman ini. Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum yang telah dilaksanakan pada tanggal 18 Mei 2026. Melalui laporan ini, penulis berharap dapat memberikan gambaran yang jelas mengenai SORTING 1.

Penulis mengucapkan terima kasih kepada Bapak Dosen serta asisten laboratorium yang telah membimbing selama praktikum berlangsung. Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari kata sempurna, baik dari segi penyusunan, bahasa, maupun kedalaman materi. Oleh karena itu, penulis dengan rendah hati menerima segala kritik dan saran yang membangun demi perbaikan dan penyempurnaan laporan di masa mendatang. Semoga laporan ini dapat bermanfaat bagi pembaca dan menjadi referensi untuk perkembangan ilmu pengetahuan di teknologi informasi.

Padang, 18 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Pratikum	1
1.3 Manfaat Pratikum	2
BAB II PEMBAHASAN	3
2.1 InsertionSort.....	3
2.2 SelectionSort	4
2.3 BubbleSort.....	6
2.4 InsertionGUI	7
BAB III KESIMPULAN	15
3.1 Kesimpulan	15
DAFTAR PUSTAKA	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengurutan data atau Sorting merupakan salah satu operasi fundamental dalam ilmu komputer dan pemrograman yang berperan penting dalam pengolahan informasi secara efisien [1]. Dalam berbagai aplikasi modern, mulai dari manajemen basis data, sistem pencarian, hingga analisis data berskala besar, algoritma pengurutan digunakan sebagai dasar untuk mengorganisasikan data sehingga proses komputasi berikutnya dapat berjalan dengan optimal. Meskipun teknologi komputasi dan perangkat keras terus berkembang, pemahaman mendalam mengenai prinsip kerja, kompleksitas waktu serta ruang dari berbagai algoritma pengurutan tetap menjadi kompetensi inti yang harus dikuasai oleh mahasiswa Department Informatika.

Praktikum Struktur Data kali ini berfokus pada implementasi tiga algoritma pengurutan berbasis perbandingan yang termasuk dalam kategori dasar, yaitu Bubble Sort, Selection Sort, dan Insertion Sort. Ketiga algoritma ini dipilih karena struktur logikanya yang relatif sederhana, sehingga sangat efektif sebagai media pembelajaran untuk memahami konsep iterasi, pertukaran data, serta analisis kompleksitas algoritma. Selain implementasi berbasis konsol, praktikum ini juga mencakup pengembangan antarmuka grafis (Graphical User Interface/GUI) untuk algoritma Insertion Sort. Penggunaan GUI bertujuan untuk memberikan visualisasi interaktif terhadap proses pengurutan, sehingga mahasiswa dapat mengamati secara langsung bagaimana data diproses langkah demi langkah dan memvalidasi kebenaran logika program [2].

1.2 Tujuan Praktikum

1. Memahami konsep dasar, prinsip kerja, dan mekanisme pertukaran data pada algoritma pengurutan berbasis perbandingan.

2. Mengimplementasikan algoritma Bubble Sort, Selection Sort, dan Insertion Sort ke dalam kode program sesuai dengan spesifikasi yang ditentukan.
3. Mengembangkan antarmuka grafis (GUI) untuk memvisualisasikan proses pengurutan secara interaktif dan memvalidasi alur eksekusi algoritma.

1.3 Manfaat Praktikum

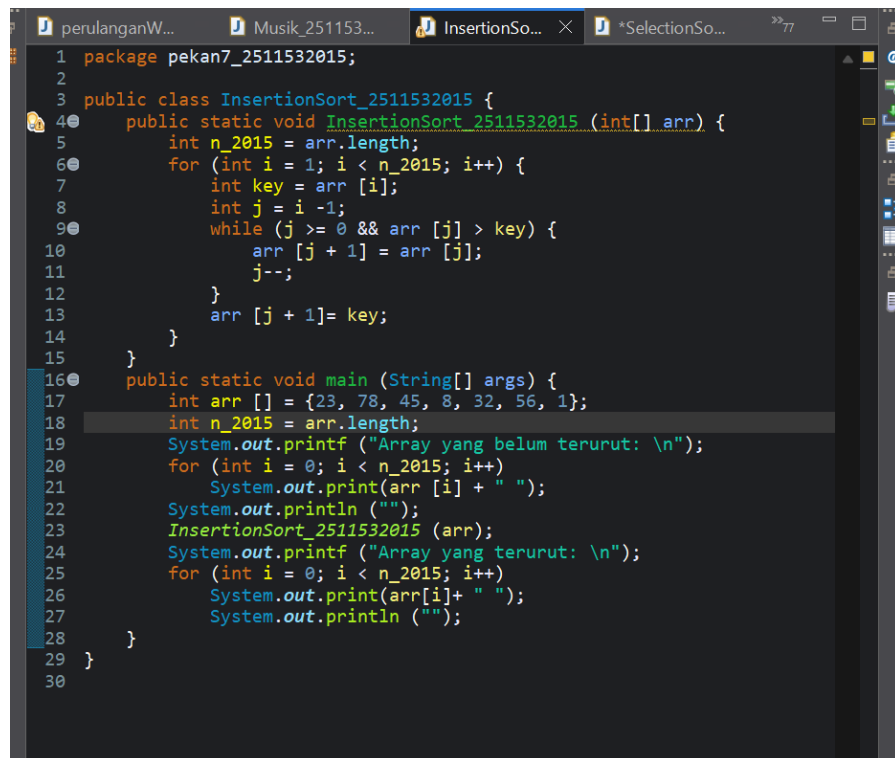
1. Meningkatkan kemampuan pemrograman, debugging, dan logika algoritma mahasiswa dalam menyelesaikan masalah pengorganisasian data secara sistematis.
2. Membekali mahasiswa dengan kemampuan visualisasi algoritma melalui GUI, yang berguna untuk pembelajaran mandiri, presentasi teknis, dan proses troubleshooting kode.
3. Memperkuat fondasi konseptual yang diperlukan untuk mempelajari topik lanjutan seperti struktru data dinamis, algoritma optimasi, dan manajemen basis data pada semester berikutnya.
4. Melatih kemampuan analisis kompratif dan dokumentasi teknis, yang merupakan kompetensi esensial dalam pengembangan pernagkat lunak professional dan riset informatika.

BAB II

PEMBAHASAN

2.1 InsertionSort_2511532015

Insertion Sort adalah algoritma pengurutan sederhana yang bekerja dengan cara menyisipkan elemen data satu per satu ke posisi yang tepat pada daftar yang sudah terurut. Algoritma ini membagi array menjadi dua bagian, yaitu bagian yang sudah terurut dan bagian yang belum terurut.



```
1 package pekan7_2511532015;
2
3 public class InsertionSort_2511532015 {
4     public static void InsertionSort_2511532015 (int[] arr) {
5         int n_2015 = arr.length;
6         for (int i = 1; i < n_2015; i++) {
7             int key = arr [i];
8             int j = i -1;
9             while (j >= 0 && arr [j] > key) {
10                 arr [j + 1] = arr [j];
11                 j--;
12             }
13             arr [j + 1]= key;
14         }
15     }
16     public static void main (String[] args) {
17         int arr [] = {23, 78, 45, 8, 32, 56, 1};
18         int n_2015 = arr.length;
19         System.out.printf ("Array yang belum terurut: \n");
20         for (int i = 0; i < n_2015; i++)
21             System.out.print(arr [i] + " ");
22         System.out.println ("");
23         InsertionSort_2511532015 (arr);
24         System.out.printf ("Array yang terurut: \n");
25         for (int i = 0; i < n_2015; i++)
26             System.out.print(arr[i]+ " ");
27         System.out.println ("");
28     }
29 }
30
```

Gambar 2.1.1: Kode program InsertionSort_2511532015

1. Inisialisasi

- Program mendeklarasikan array dengan 7 elemen
- Variable n_2015 menyimpan panjang array

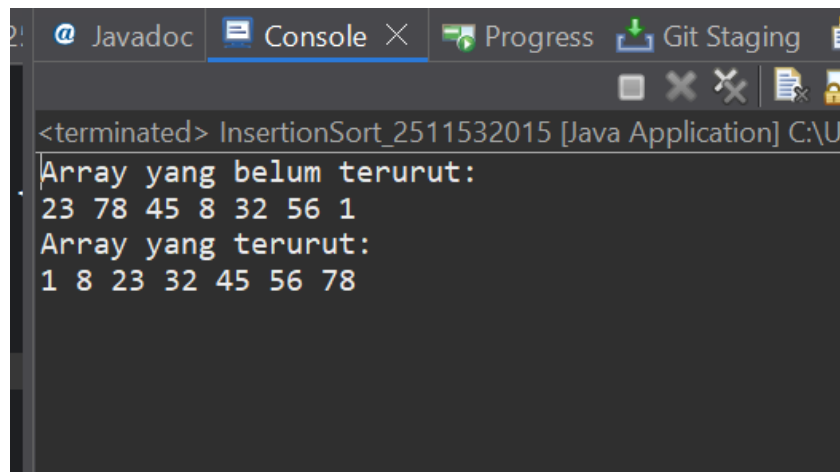
2. Loop

- Loop for dimulai dari indeks $i = 1$ hingga $n_2015 - 1$
- Pada setiap iterasi, elemen $arr[i]$ disimpan dalam variable key
- Variable j diinisialisasikan dengan nilai $i - 1$ untuk membandingkan elemen sebelumnya.

- Loop while menjalankan proses: membandingkan `arr[j]` dengan `key`, jika `arr[j] > key`, maka `arr[j]` digeser satu posisi ke kanan, nilai `j` dikurangi 1 (`j--`).
- Setelah posisi yang tepat ditemukan, `key` ditempatkan pada `arr[j+1]`.

3. Output

Program menampilkan array sebelum dan sesudah pengurutan.



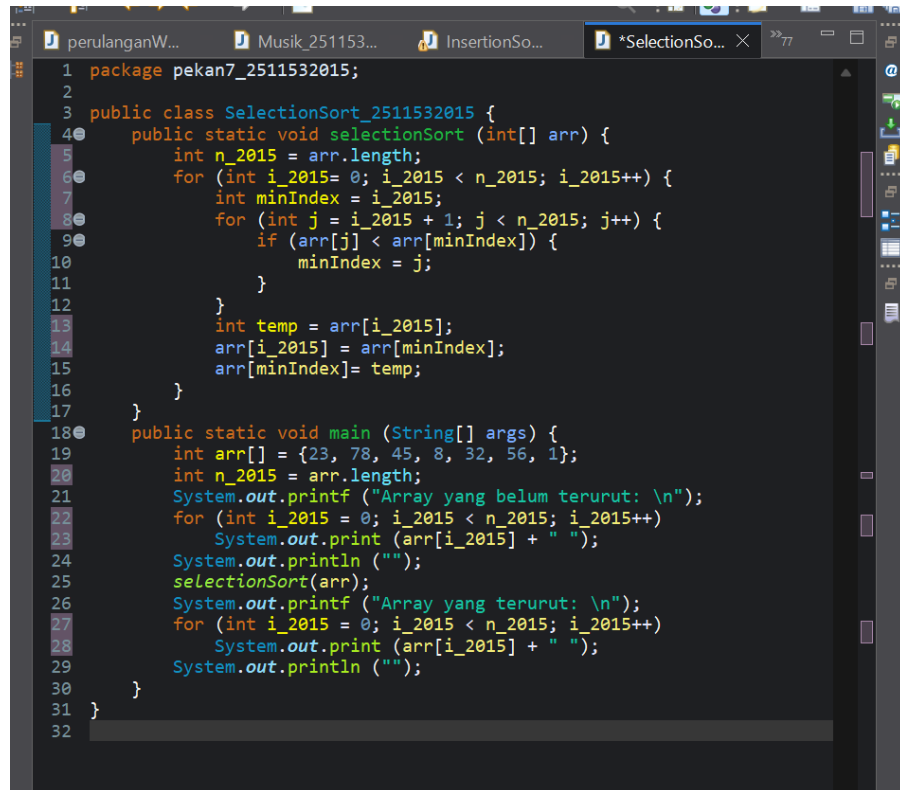
```
<terminated> InsertionSort_2511532015 [Java Application] C:\U
Array yang belum terurut:
23 78 45 8 32 56 1
Array yang terurut:
1 8 23 32 45 56 78
```

Gambar 2.1.2: Hasi output kode program InsertionSort_2511532015

Berdasarkan output, algoritma Insertion Sort berhasil mengurutkan array dari nilai terkecil ke terbesar, yang menunjukkan algoritma ini cocok untuk data dalam jumlah kecil karena implementasinya yang sederhana, namun kurang efisien untuk data berskala besar.

2.2 SelectionSort_2511532015

Selection sort adalah algoritma pengurutan sederhana yang bekerja dengan cara mencari elemen terkecil (atau terbesar) dari sekumpulan data, lalu menempatkannya pada posisi yang tepat.



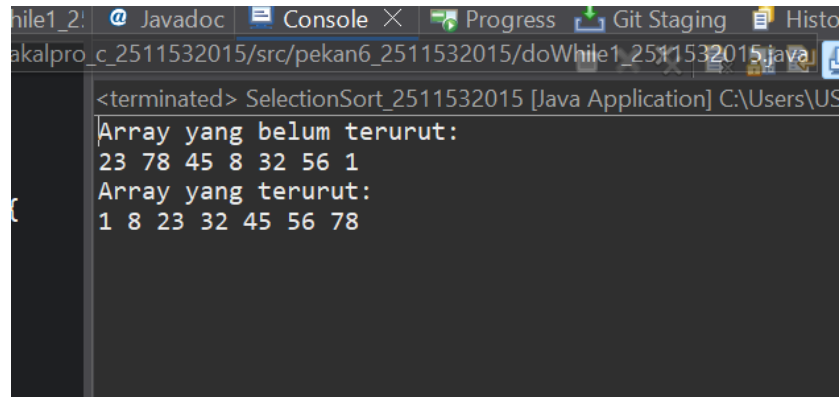
```
1 package pekan7_2511532015;
2
3 public class SelectionSort_2511532015 {
4     public static void selectionSort (int[] arr) {
5         int n_2015 = arr.length;
6         for (int i_2015= 0; i_2015 < n_2015; i_2015++) {
7             int minIndex = i_2015;
8             for (int j = i_2015 + 1; j < n_2015; j++) {
9                 if (arr[j] < arr[minIndex]) {
10                     minIndex = j;
11                 }
12             }
13             int temp = arr[i_2015];
14             arr[i_2015] = arr[minIndex];
15             arr[minIndex]= temp;
16         }
17     }
18     public static void main (String[] args) {
19         int arr[] = {23, 78, 45, 8, 32, 56, 1};
20         int n_2015 = arr.length;
21         System.out.printf ("Array yang belum terurut: \n");
22         for (int i_2015 = 0; i_2015 < n_2015; i_2015++)
23             System.out.print (arr[i_2015] + " ");
24         System.out.println ("");
25         selectionSort(arr);
26         System.out.printf ("Array yang terurut: \n");
27         for (int i_2015 = 0; i_2015 < n_2015; i_2015++)
28             System.out.print (arr[i_2015] + " ");
29         System.out.println ("");
30     }
31 }
32
```

Gambar 2.2.1: Kode Program SelectionSort_2511532015

1. Pertama kode ini mendefenisikan fungsi, lalu menghitung berapa banyak angka dalam array.
2. Loop
 - Perulangan for pertama menentukan posisi mana yang akan diisi dengan angka terkecil lalu diasumsikan angka di posisi i_2015 adalah yang terkecil (minIndex = i_2015).
 - Perulangan for yang kedua untuk mencari angka terkecil, j adalah variabel untuk melihat ke sisa angka di belakang, jika kita angka di posisi j tersebut lebih kecil daripada angka minIndex maka update posisi terkecil ke j (minIndex = j).
3. Penukaran (swap)

minIndex sekarang menunjuk ke angka terkecil yang sebenarnya. Maka, tukar angka di posisi saat ini (i_2015) dengan angka terkecil (minIndex) menggunakan variabel bantuan temp.
4. Output

Program membuat array lalu fungsi selectionSort dipanggil, maka akan menampilkan output:

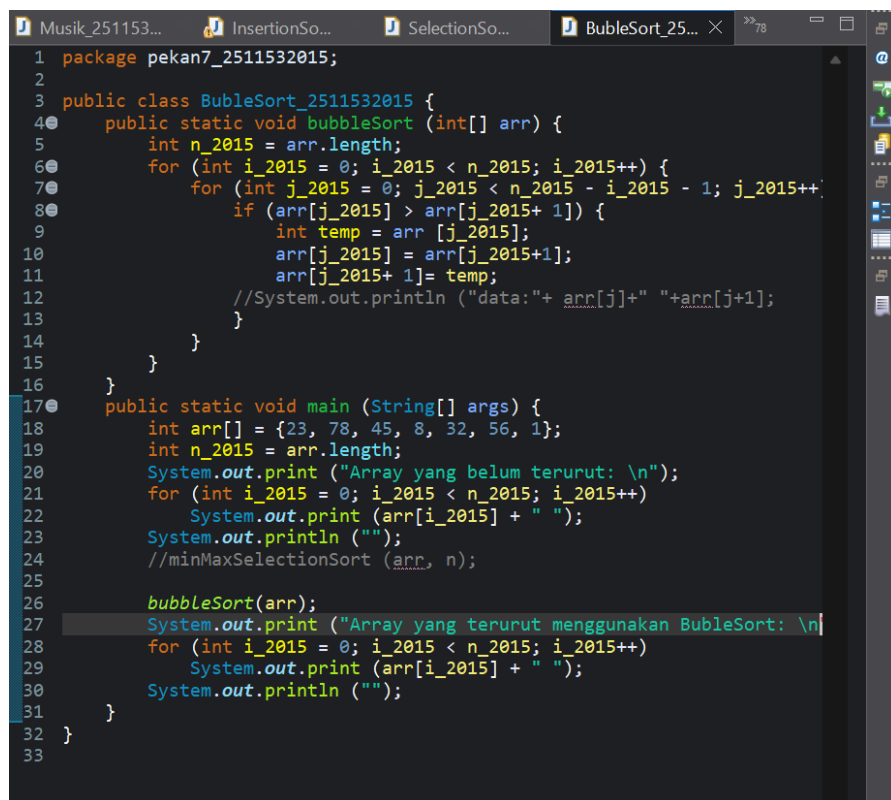


```
<terminated> SelectionSort_2511532015 [Java Application] C:\Users\US
Array yang belum terurut:
23 78 45 8 32 56 1
Array yang terurut:
1 8 23 32 45 56 78
```

Gambar 2.2.2: Hasil output kode program SelectionSort_2511532015

2.3 BubleSort_2511532015

Bubble sort adalah algoritma pengurutan data paling sederhana yang bekerja dengan cara membandingkan dua elemen yang bersebalahan dan menukarnya jika posisinya salah.



```
1 package pekan7_2511532015;
2
3 public class BubleSort_2511532015 {
4     public static void bubbleSort (int[] arr) {
5         int n_2015 = arr.length;
6         for (int i_2015 = 0; i_2015 < n_2015; i_2015++) {
7             for (int j_2015 = 0; j_2015 < n_2015 - i_2015 - 1; j_2015++) {
8                 if (arr[j_2015] > arr[j_2015+ 1]) {
9                     int temp = arr [j_2015];
10                    arr[j_2015] = arr[j_2015+1];
11                    arr[j_2015+ 1]= temp;
12                    //System.out.println ("data: "+ arr[j]+" "+arr[j+1];
13                }
14            }
15        }
16    }
17    public static void main (String[] args) {
18        int arr[] = {23, 78, 45, 8, 32, 56, 1};
19        int n_2015 = arr.length;
20        System.out.print ("Array yang belum terurut: \n");
21        for (int i_2015 = 0; i_2015 < n_2015; i_2015++)
22            System.out.print (arr[i_2015] + " ");
23        System.out.println ("");
24        //minMaxSelectionSort (arr, n);
25
26        bubbleSort(arr);
27        System.out.print ("Array yang terurut menggunakan BubleSort: \n");
28        for (int i_2015 = 0; i_2015 < n_2015; i_2015++)
29            System.out.print (arr[i_2015] + " ");
30        System.out.println ("");
31    }
32 }
33 }
```

Gambar 2.3.1: Kode Program BubleSort_2511532015

1. Membuat nama fungsi yang menerima arr dan menghitung panjangnya dalam n_2015.
2. Loop

- Loop pertama menentukan berapa kali menyisir array, semakin banyak putaran (i) semakin banyak data di akhir array yang sudah terurut dengan benar.
 - Loop kedua proses membandingkan, j_2015 mulai dari 0 dengan batas loop adalah $n_2015 - i_2015 - 1$, karena setiap selesai satu putaran satu angka terbesar pasti ada di posisi terakhir jadi tidak perlu membandingkan angka yang sudah ada di akhir, dikurang 1 karena di dalam loop akan dibandingkan j dengan j+1, jika j sudah sampai elemen terakhir j+1 akan error (keluar batas array), jadi berhenti satu langkah lebih awal.
3. Penukaran, dimana cek apakah angka di kiri (j) lebih besar dari angka di kanan (j+1), jika ya maka lakukan penukaran menggunakan variable temp dan angka besar digeser ke kanan, jika tidak maka biarkan saja lalu lanjut ke perbandingan berikutnya.
 4. Program membuat array dan mencetaknya sebelum diurutkan lalu method bubbleSort dipanggil lalu dicetak array hasil pengurutan, akan tampak seperti berikut outputnya:

```
<terminated> BubleSort_2511532015 [Java Application] C:\Users...
Array yang belum terurut:
23 78 45 8 32 56 1
Array yang terurut menggunakan BubleSort:
1 8 23 32 45 56 78
```

Gambar 2.3.2: Hasil output kode program bubbleSort_2511532015

2.4 InsertionGUI_2511532015

```
package pekan7_2511532015;

import java.awt.BorderLayout;
```

```

import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;
import java.lang.reflect.Array;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;

public class InsertionGUI_2511532015 extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private int[] array_2015;
    private JLabel[] labelArray_2015;
    private JButton stepButton_2015, resetButton_2015,
    setButton_2015;
    private JTextField inputField_2015;
    private JPanel panelArray_2015;
    private JTextArea stepArea_2015;

    private int i_2015 = 1, j_2015;
    private boolean sorting_2015 = false;
    private int stepCount_2015 = 1;

    /**
     * Create the frame.
     */
    public InsertionGUI_2511532015() {
        setTitle ("Insertion Sort Langkah per Langkah");
        setSize (750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo (null);
        setLayout (new BorderLayout());

        //Panel Input
        JPanel inputPanel = new JPanel (new FlowLayout());
        inputField_2015 = new JTextField (30);
        setButton_2015 = new JButton ("Set Array");
        inputPanel.add(new JLabel ("Masukkan angka (pisahkan dengan
koma): "));
        inputPanel.add(inputField_2015);
        inputPanel.add(setButton_2015);

        //Panel array visual

```

```

panelArray_2015 = new JPanel ();
panelArray_2015.setLayout (new FlowLayout());

//panel kontrol
JPanel controlPanel = new JPanel();
setButton_2015 = new JButton ("Langkah selanjutnya");
resetButton_2015 = new JButton ("Reset");
stepButton_2015 = new JButton ("Reset");
controlPanel.add(stepButton_2015);
controlPanel.add(resetButton_2015);

//area teks untuk log langkah-langkah
stepArea_2015 = new JTextArea (8, 60);
stepArea_2015.setEditable (false);
stepArea_2015.setFont (new Font ("Monospace", Font.PLAIN,
14));
JScrollPane scrollPane = new JScrollPane (stepArea_2015);

//Tambahkan panel ke frame
add(inputPanel, BorderLayout.NORTH);
add(panelArray_2015, BorderLayout.CENTER);
add(controlPanel, BorderLayout.SOUTH);
add(scrollPane, BorderLayout.EAST);

//Event Set Array
setButton_2015.addActionListener (e ->
setArrayFromInput());

//Event Langkah Selanjutnya
stepButton_2015.addActionListener (e -> performStep());

//Event Reset
resetButton_2015.addActionListener (e -> reset());
}
private void setArrayFromInput() {
    String text = inputField_2015.getText().trim();
    if (text.isEmpty()) return;
    String[] parts = text.split (",");
    array_2015 = new int [parts.length];
    try {
        for (int k = 0; k < parts.length; k++) {
            array_2015 [k] =
Integer.parseInt(parts[k].trim());
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Masukkan hanya
angka yang dipisahkan "
+ "dengan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
        return;
    }
    i_2015 = 1;
    stepCount_2015 = 1;
    sorting_2015 = true;
    stepButton_2015.setEnabled(true);
    stepArea_2015.setText("");
    panelArray_2015.removeAll();
}

```

```

        labelArray_2015 = new JLabel [array_2015.length];
        for (int k = 0; k < array_2015.length; k++) {
            labelArray_2015[k] = new JLabel (String.valueOf
(array_2015[k]));
            labelArray_2015[k].setFont (new Font ("Arial",
Font.BOLD, 24));

            labelArray_2015[k].setBorder(BorderFactory.createLineBorder
(Color.BLACK));
            labelArray_2015[k].setPreferredSize(new Dimension
(50, 50));

            labelArray_2015[k].setHorizontalAlignment(SwingConstants.CE
NTER);
            panelArray_2015.add(labelArray_2015[k]);
        }
        panelArray_2015.revalidate();
        panelArray_2015.repaint();
    }
    private void performStep() {
        if (i_2015 < array_2015.length && sorting_2015) {
            int key = array_2015[i_2015];
            j_2015 = i_2015 - 1;

            StringBuilder stepLog = new StringBuilder();
            stepLog.append("Langkah ").append(stepCount_2015).
append(": Memasukkan ").append(key).append ("\\n");

            while (j_2015 >= 0 && array_2015[j_2015] > key) {
                array_2015[j_2015 + 1] = array_2015[j_2015];
                j_2015--;
            }
            array_2015[j_2015 + 1] = key;

            updateLabels();
            stepLog.append("Hasil:
").append(arrayToString(array_2015)).
append("\\n\\n");
            stepArea_2015.append(stepLog.toString());

            i_2015++;
            stepCount_2015++;

            if (i_2015 == array_2015.length) {
                sorting_2015 = false;
                stepButton_2015.setEnabled(false);
                JOptionPane.showMessageDialog (this, "Sorting
selesai!");
            }
        }
    }
    private void updateLabels() {
        for (int k = 0; k < array_2015.length; k++) {

            labelArray_2015[k].setText(String.valueOf(array_2015[k]));
        }
    }
}

```

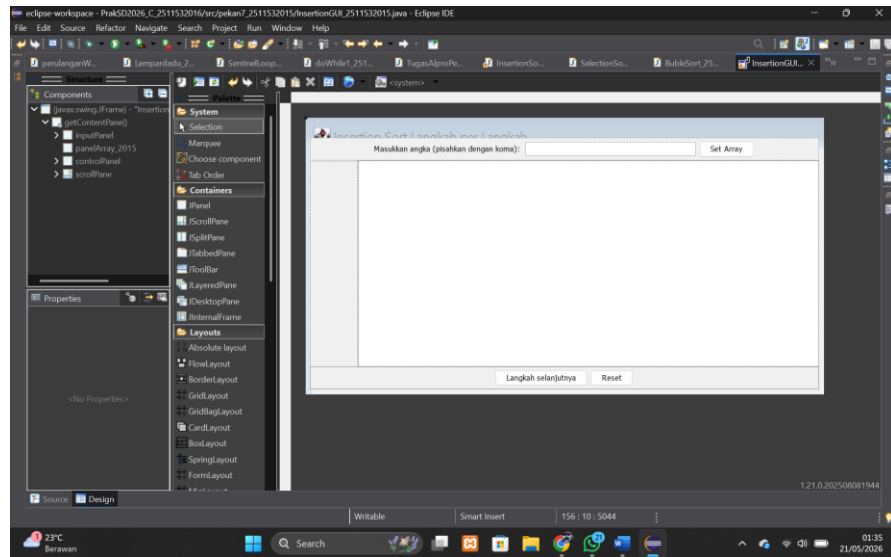
```

private void reset() {
    inputField_2015.setText("");
    panelArray_2015.removeAll();
    panelArray_2015.revalidate();
    panelArray_2015.repaint();
    stepArea_2015.setText("");
    sorting_2015 = false;
    i_2015 = 1;
    stepCount_2015 = 1;
}
private String arrayToString (int[] arr) {
    StringBuilder sb = new StringBuilder();
    for (int k = 0; k < arr.length; k++) {
        sb.append (arr[k]);
        if (k < arr.length - 1) sb.append (" , ");
    }
    return sb.toString();
}
public static void main (String[] args) {
    SwingUtilities.invokeLater(() -> {
        InsertionSort_2511532015 gui = new
        InsertionSort_2511532015();
        gui.setVisible(true);
    });
}
}

```

Program ini adalah implementasi Insertion Sort yang dilengkapi dengan antarmuka grafis (GUI) menggunakan library Java Swing. Dalam source code ini memiliki beberapa method, yaitu:

1. InsertionGUI_2015(): membangun tampilan GUI seperti panel input, visual array, tombol kontrol, dan area log.
2. setArrayInput(): mengambil input teks lalu di parse jadi array integer kemudian ditampilkan sebagai kotak visual.
3. performStep(): menjadlankan 1 langkah Insertion Sort
4. updateLabels(): memperbarui teks pada kotak visual agar sesuai dengan data terbaru.
5. reset(): mengosongkan input, visual, dan log
6. arrayToString(): mengubah array integer jadi string rapi untuk ditampilkan di log
7. main(): membuat dan menampilkan jendela GUI.

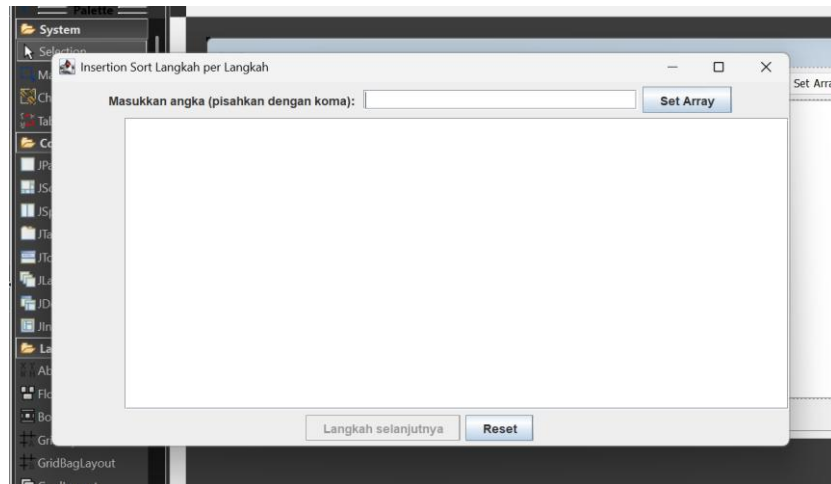


Gambar 2.4.1: Design InsertionGUI_2511532015

Design InsertionGUI ini mempunyai beberapa elemen, yaitu:

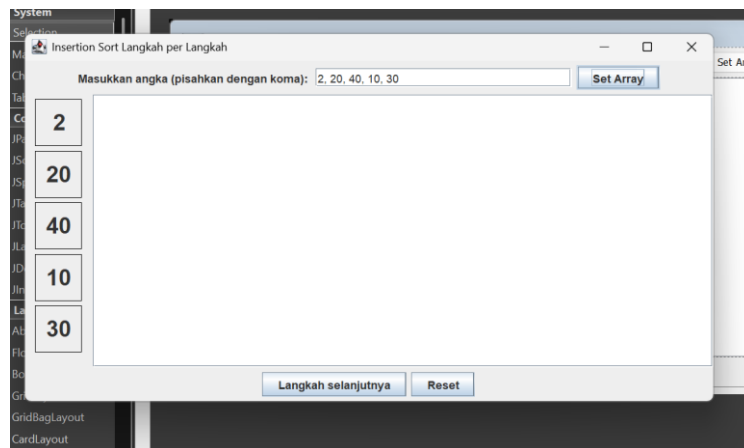
1. Bagian atas ada kotak kosong JTextField untuk tempat input angka lalu ada JButton “Set Array”
2. Bagian kiri terdapat panel kosong yang berfungsi sebagai area visualisasi elemen array. Panel ini menggunakan FlowLayout sehingga kotak-kotak angka tersusun secara horizontal dengan spacing otomatis.
3. Bagian bawah ada panel kontrol yang terdiri dari: JButton “Langkah Selanjutnya” untuk mengeksekusi satu iterasi algoritma Insertion Sort, dan JButton “Reset” untuk mereset seluruh komponen ke kondisi awal.
4. Area log langkah pada bagian tengah untuk menampilkan catatan proses pengurutan secara real-time, dan juga memungkinkan scrolling pada JTextArea ketika konten melebihi kapasitas tampilan.

Ketika dijalankan program akan menghasilkan output:



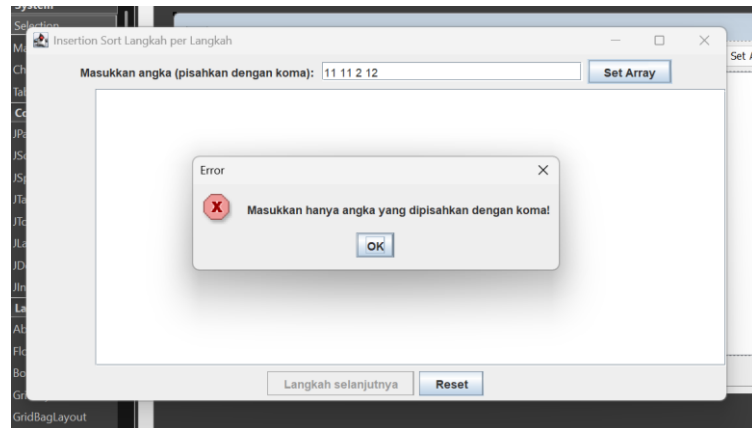
Gambar 2.4.2: Tampilan awal output program InsertionGUI_2511532015

Saat program pertama kali dijalankan akan gambar seperti diatas, kolom input masih kosong, area tengah putih juga masih kosong, dan tombol “Langkah Selanjutnya” tidak bisa diklik.

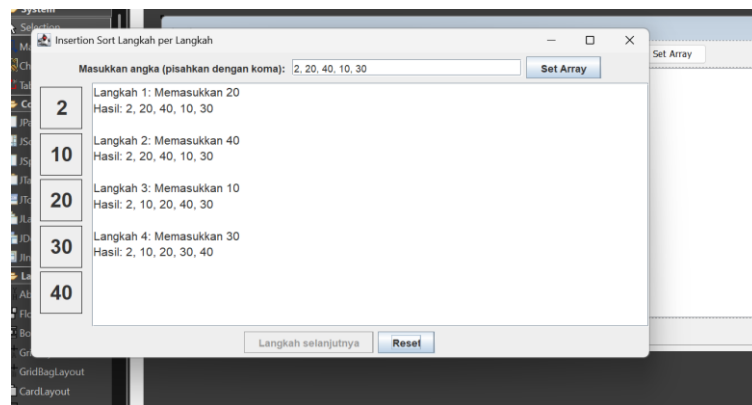


Gambar 2.4.3: Proses input angka oleh user

Di sini pengguna akan menginputkan angka yang dipisahkan oleh koma, tekan set array untuk mulai memproses pengurutan angka menggunakan Insertion Sort.

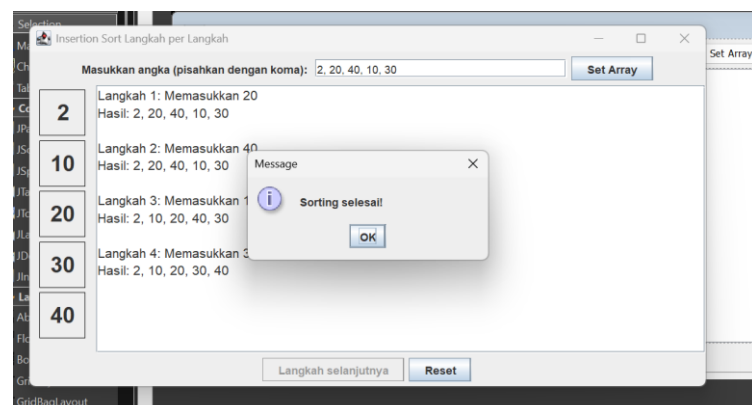


Gambar 2.4.4: Input pengguna jika tidak dipisahkan dengan koma
Akan muncul error message jika pengguna menginputkan angka tanpa dipisahah dengan koma.



Gambar 2.4.5: Proses Iterasi

Proses pengurutan akan dilakukan satu persatu dengan menekan langkah selanjutnya, hingga proses pengurutan selesai yang ditandai dengan munculnya pesan seperti gambar di bawah.



Gambar 2.4.6: Pesan jika proses pengurutan telah selesai

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan hasil praktikum, dapat disimpulkan bahwa algoritma pengurutan dasar (Bubble Sort, Selection Sort, dan Insertion Sort) berhasil diimplementasikan menggunakan bahasa pemrograman Java. Ketiga algoritma tersebut termasuk dalam kategori comparison-based sorting dengan kompleksitas waktu $O(n^2)$, namun memiliki karakteristik yang berbeda: Insertion Sort lebih efisien untuk data kecil atau hampir terurut, Selection Sort meminimalkan jumlah operasi penukaran, sedangkan Bubble Sort paling sederhana dalam logika perbandingan bersebelahan.

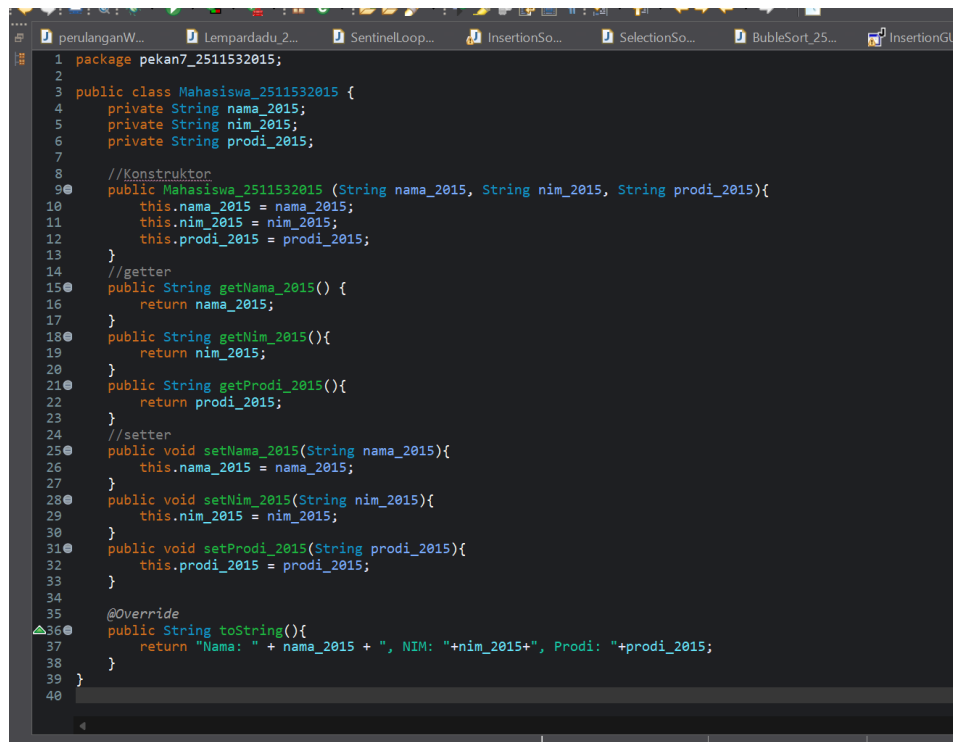
Selain itu, pengembangan antarmuka grafis (GUI) pada Insertion Sort berhasil memvisualisasikan proses pengurutan secara interaktif, sehingga memudahkan pemahaman alur algoritma, pemantauan perubahan data, serta validasi logika program per langkah. Praktikum ini tidak hanya memperkuat kemampuan penulisan kode dan debugging, tetapi juga memberikan pemahaman praktis mengenai analisis efisiensi algoritma, yang menjadi fondasi penting untuk mempelajari mata kuliah struktur data dan algoritma lanjutan.

DAFTAR PUSTAKA

- [1] Z. N. Hanilatifa, "Struktur Data: Sorting," Medium, 25 Maret 2025. [Online]. Available: <https://zahrahani.medium.com/struktur-data-sorting-6a30fe60684f>. [Accessed 20 May 2026].
- [2] W. E. Pratama, M. I. Muhair and Y. Servanda, "Comparison of the Efficiency of Bubble Sort and Insertion Sort Algorithms in Sorting Sales Data in Digital Printing," *Journal of Artificial Intelligence and Engineering Applications*, vol. 5, no. 2026, 2026.

PENJELASAN TUGAS PRAKTIKUM PEKAN 7

1. Kode Program kelas ADT Mahasiswa



```
1 package pekan7_2511532015;
2
3 public class Mahasiswa_2511532015 {
4     private String nama_2015;
5     private String nim_2015;
6     private String prodi_2015;
7
8     //Konstruktor
9     public Mahasiswa_2511532015 (String nama_2015, String nim_2015, String prodi_2015){
10         this.nama_2015 = nama_2015;
11         this.nim_2015 = nim_2015;
12         this.prodi_2015 = prodi_2015;
13     }
14     //getter
15     public String getNama_2015() {
16         return nama_2015;
17     }
18     public String getNim_2015(){
19         return nim_2015;
20     }
21     public String getProdi_2015(){
22         return prodi_2015;
23     }
24     //setter
25     public void setNama_2015(String nama_2015){
26         this.nama_2015 = nama_2015;
27     }
28     public void setNim_2015(String nim_2015){
29         this.nim_2015 = nim_2015;
30     }
31     public void setProdi_2015(String prodi_2015){
32         this.prodi_2015 = prodi_2015;
33     }
34
35     @Override
36     public String toString(){
37         return "Nama: " + nama_2015 + ", NIM: "+nim_2015+", Prodi: "+prodi_2015;
38     }
39 }
40
```

Struktur kode program terbagi menjadi tiga komponen utama. Pertama, kelas ADT Mahasiswa_2511532015 berfungsi sebagai model data yang menyimpan atribut nama, NIM, dan prodi, dilengkapi dengan constructor, getter/setter, serta method toString() untuk format tampilan.

2. Source Code MahasiswaGUI_2511532015

```
package pekan7_2511532015;

import java.awt.BorderLayout;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.util.ArrayList;

import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
```

```

public class MahasiswaGUI_2511532015 extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private JTextField inputNama_2015, inputNim_2015,
    inputProdi_2015;
    private JButton tambahButton_2015, hapusButton_2015,
    sortingButton_2015;
    private JComboBox<String> pilihAlgoritma_2015;
    private JTable tableData_2015;
    private DefaultTableModel tableModel_2015;
    private JTextArea logText_2015;
    private ArrayList<Mahasiswa_2511532015>
    daftarMahasiswa_2015;

    /**
     * Launch the application.
     */
    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    MahasiswaGUI_2511532015 frame = new
MahasiswaGUI_2511532015();
                    frame.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }

    /**
     * Create the frame.
     */
    public MahasiswaGUI_2511532015() {
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setBounds(100, 100, 608, 480);
        contentPane = new JPanel();
        contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
        setContentPane(contentPane);

        //inisialisasi arraylist
        daftarMahasiswa_2015 = new ArrayList<>();
        contentPane.setLayout(new BorderLayout(10, 10));

        //panel input
        JPanel inputPanel_2015 = new JPanel (new FlowLayout());
        inputNama_2015 = new JTextField (15);
        inputNim_2015 = new JTextField (15);
        inputProdi_2015 = new JTextField (15);
        tambahButton_2015 = new JButton ("Tambah");
        hapusButton_2015 = new JButton ("Hapus");
        contentPane.add(inputPanel_2015, BorderLayout.NORTH);

        inputPanel_2015.add(new JLabel("Nama:"));
        inputPanel_2015.add(inputNama_2015);
    }
}

```

```

inputPanel_2015.add(new JLabel("NIM:"));
inputPanel_2015.add(inputNim_2015);

inputPanel_2015.add(new JLabel("Prodi:"));
inputPanel_2015.add(inputProdi_2015);

inputPanel_2015.add(tambahButton_2015);
inputPanel_2015.add(hapusButton_2015);

//table
String[] kolom_2015 = {"Nama", "NIM", "Prodi"};
tableModel_2015 = new DefaultTableModel (kolom_2015, 0);
tableData_2015 = new JTable (tableModel_2015);
tableData_2015.setFillsViewportHeight(true);
JScrollPane scrollTable_2015 = new JScrollPane
(tableData_2015);
contentPane.add(scrollTable_2015, BorderLayout.CENTER);

//panel kontrol
JPanel panelKontrol_2015 = new JPanel (new FlowLayout());
panelKontrol_2015.add(new JLabel ("Algoritma: "));
pilihAlgoritma_2015 = new JComboBox<>(new String[] {
    "Insertion Sort", "Selection Sort", "Bubble
Sort"
});
panelKontrol_2015.add(pilihAlgoritma_2015);
sortingButton_2015 = new JButton ("Mulai Sorting");
panelKontrol_2015.add(sortingButton_2015);
contentPane.add(panelKontrol_2015, BorderLayout.SOUTH);

//area log
logText_2015 = new JTextArea (40, 55);
logText_2015.setEditable(false);
logText_2015.setFont(new java.awt.Font("Monospaced",
java.awt.Font.PLAIN, 12));
JScrollPane scrollLog_2015 = new JScrollPane
(logText_2015);
contentPane.add(scrollLog_2015, BorderLayout.EAST);

//tombol tambah
tambahButton_2015.addActionListener(e-> {
    String nama= inputNama_2015.getText().trim();
    String nim= inputNim_2015.getText().trim();
    String prodi= inputProdi_2015.getText().trim();

    if (nama.isEmpty() ||
nim.isEmpty() || prodi.isEmpty()) {
        javax.swing.JOptionPane.showMessageDialog
(this, "Semua field harus diisi!", "Error",
        javax.swing.JOptionPane.ERROR_MESSAGE);
        return;
    }
    Mahasiswa_2511532015 mhsBaru = new
Mahasiswa_2511532015(nama, nim, prodi);
daftarMahasiswa_2015.add(mhsBaru);

```

```

        tambahKeTable_2015(mhsBaru);
        inputNama_2015.setText("");
        inputNim_2015.setText("");
        inputProdi_2015.setText("");
        inputNama_2015.requestFocus();
        tambahLog_2015("Data ditambahkan: "+nama);
    });

    //tombol hapus
    hapusButton_2015.addActionListener(e->{
        int selectedRow = tableData_2015.getSelectedRow();
        if (selectedRow == -1) {

            javax.swing.JOptionPane.showMessageDialog(this, "Pilih data
yang ingin dihapus!", "Peringatan",
                javax.swing.JOptionPane.WARNING_MESSAGE);
        }
        daftarMahasiswa_2015.remove(selectedRow);
        tableModel_2015.removeRow(selectedRow);
        tambahLog_2015 ("Data dihapus dari baris "+
(selectedRow + 1));
    });

    //tombol mulai sorting
    sortingButton_2015.addActionListener(e->{
        if (daftarMahasiswa_2015.isEmpty()) {

            javax.swing.JOptionPane.showMessageDialog(this, "Tamah data
terlebih dahulu", "Peringatan",

            javax.swing.JOptionPane.WARNING_MESSAGE);
            return;
        }
        String pilih =
(String)pilihAlgoritma_2015.getSelectedItem();
        ArrayList<Mahasiswa_2511532015> dataSorting = new
ArrayList<>(daftarMahasiswa_2015);
        logText_2015.setText("");
        tambahLog_2015("Memulai "+pilih);

        switch (pilih) {
            case "Insertion Sort":
                insertionSort_2015 (dataSorting);
                break;
            case "Selection Sort":
                selectionSort_2015(dataSorting);
                break;
            case "Bubble Sort":
                bubbleSort_2015(dataSorting);
                break;
        }
        tambahLog_2015 ("Sorting selesai");
        javax.swing.JOptionPane.showMessageDialog(this,
"Sorting selesai!");
    });
}

```

```

//menambahkan objek mahasiswa ke table
private void tambahKeTable_2015 (Mahasiswa_2511532015 mhs) {
    tableModel_2015.addRow(new Object[] {
        mhs.getNama_2015(),
        mhs.getNim_2015(),
        mhs.getProdi_2015()
    });
}

// menambahkan log
private void tambahLog_2015(String pesan) {
    logText_2015.append(pesan + "\n");

    logText_2015.setCaretPosition(logText_2015.getDocument().getLength());
}

public void updateTable_2015() {
    tableModel_2015.setRowCount(0);
    for (Mahasiswa_2511532015 mhs : daftarMahasiswa_2015) {
        tambahKeTable_2015(mhs);
    }
}

//update table
private void
updateTabelDariList_2015(ArrayList<Mahasiswa_2511532015>
data) {
    tableModel_2015.setRowCount(0);
    for (Mahasiswa_2511532015 mhs : data) {
        tableModel_2015.addRow(new Object[]{
            mhs.getNama_2015(),
            mhs.getNim_2015(),
            mhs.getProdi_2015()
        });
    }
}

//konversi ArrayList ke string untuk log
private String
arrayToString_2015(ArrayList<Mahasiswa_2511532015> data) {
    StringBuilder sb = new StringBuilder("[");
    for (int i = 0; i < data.size(); i++) {
        sb.append(data.get(i).getNama_2015());
        if (i < data.size() - 1) sb.append(", ");
    }
    sb.append("]");
    return sb.toString();
}

//Insertion Sort
private void
insertionSort_2015(ArrayList<Mahasiswa_2511532015> data) {
    int n = data.size();
    tambahLog_2015("=== INSERTION SORT ===");

    for (int i = 1; i < n; i++) {
        Mahasiswa_2511532015 key = data.get(i);
        int j = i - 1;

        // Geser elemen yang lebih besar

```



```

        while (j >= 0 &&
data.get(j).getNama_2015().compareToIgnoreCase(key.getNama_2
015()) > 0) {
            data.set(j + 1, data.get(j));
            j--;
        }
        data.set(j + 1, key);

        tableModel_2015.fireTableDataChanged();
        tableData_2015.repaint();
        logText_2015.repaint();
        tambahLog_2015("Langkah " + i + ": " +
arrayToString_2015(data));
    }
}
//Selection Sort
private void
selectionSort_2015(ArrayList<Mahasiswa_2511532015> data) {
    int n = data.size();
    int pass = 1;
    tambahLog_2015("=== SELECTION SORT ===");

    for (int i = 0; i < n - 1; i++) {
        // Cari indeks elemen terkecil di sisa array
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            // Bandingkan menggunakan compareToIgnoreCase()
            if
(data.get(j).getNama_2015().compareToIgnoreCase(
                data.get(minIndex).getNama_2015()) < 0) {
                minIndex = j;
            }
        }

        // Tukar elemen terkecil dengan posisi i
        Mahasiswa_2511532015 temp = data.get(i);
        data.set(i, data.get(minIndex));
        data.set(minIndex, temp);
        tableModel_2015.fireTableDataChanged();
        tableData_2015.repaint();
        logText_2015.repaint();
        tambahLog_2015("Pass " + pass + ": " +
arrayToString_2015(data));
    }
}
//Bubble Sort
private void bubbleSort_2015(ArrayList<Mahasiswa_2511532015>
data) {
    int n = data.size();
    int pass = 1;
    tambahLog_2015("=== BUBBLE SORT ===");

    for (int i = 0; i < n - 1; i++) {
        // Bandingkan elemen bersebelahan
        for (int j = 0; j < n - i - 1; j++) {
            // Jika elemen j > elemen j+1, swap

```

```

        if
        (data.get(j).getNama_2015().compareToIgnoreCase(
            data.get(j + 1).getNama_2015()) > 0) {

            // Swap elemen
            Mahasiswa_2511532015 temp = data.get(j);
            data.set(j, data.get(j + 1));
            data.set(j + 1, temp);

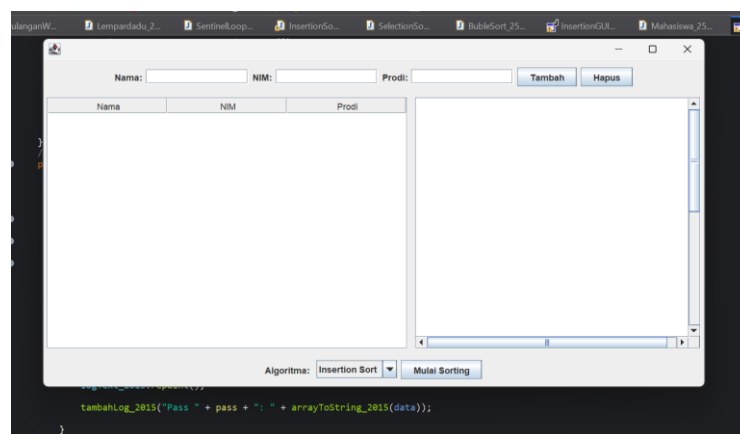
        }
    }
    tableModel_2015.fireTableDataChanged();
    tableData_2015.repaint();
    logText_2015.repaint();

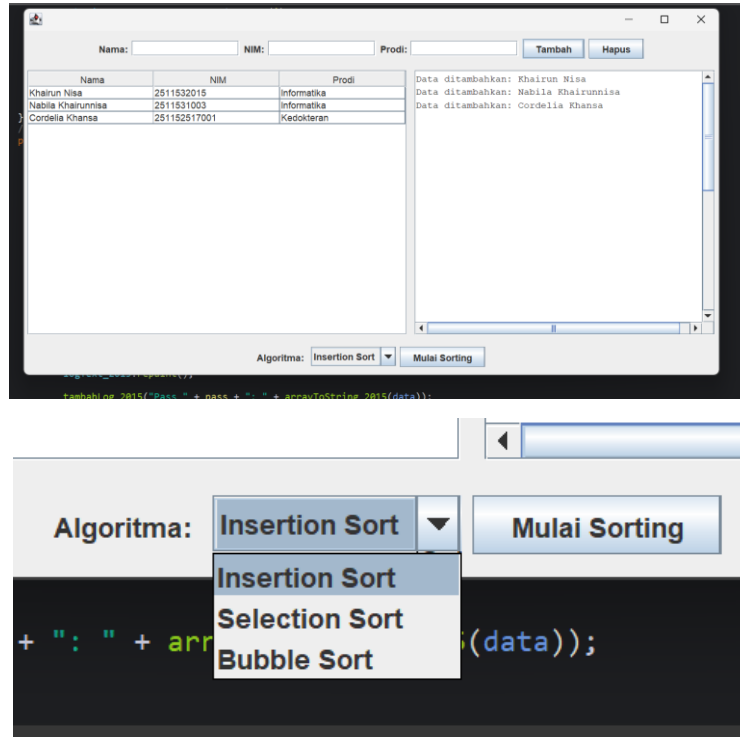
    tambahLog_2015("Pass " + pass + ": " +
        arrayToString_2015(data));
    }
}

```

MahasiswaGUI_2511532015 dibangun menggunakan library javax.swing dengan tata letak BorderLayout. Kelas ini menangani antarmuka pengguna, validasi input, manajemen data berbasis ArrayList, serta pengaturan event listener pada tombol dan JComboBox. Ketiga, terdapat tiga method sorting terpisah (insertionSort, selectionSort, bubbleSort) yang dikhususkan untuk mengolah objek Mahasiswa. Seluruh perbandingan string menggunakan method compareToIgnoreCase() agar tidak sensitif terhadap huruf kapital, dan setiap perubahan data pada iterasi algoritma secara otomatis memicu pembaruan visual pada tabel serta pencatatan log langkah pada area teks.

3. Output MahasiswaGUI_2511532015





Output program berupa antarmuka grafis (GUI) yang menampilkan tabel data mahasiswa di area tengah serta panel log di sisi kanan. Setelah data diinput dan algoritma dipilih melalui ComboBox, proses pengurutan divisualisasikan secara interaktif langkah demi langkah menggunakan tombol "Langkah Selanjutnya", di mana setiap klik secara otomatis memperbarui susunan nama pada tabel dan mencatat status iterasi pada area log dengan format Langkah X: atau Pass X:. Seluruh perbandingan string dilakukan menggunakan method `compareToIgnoreCase()` agar pengurutan bersifat tidak sensitif terhadap huruf kapital. Proses berlanjut secara bertahap hingga seluruh data tersusun secara alfabetis (ascending A–Z), ditandai dengan notifikasi `=== Sorting Selesai ===` pada log dan penonaktifan tombol langkah, sehingga alur kerja masing-masing algoritma dapat dipantau secara transparan dan terkontrol oleh pengguna.